

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development | **Course Code:** 23CM4121

Observation Task No.: 5 | **Question No.:** 3

Student Name: J Satya Srikar | **Roll No.:** A24126552259 | **Class:** CSM-B

Date: 22/09/2026 | **File Creation Time:** 22-09-2026 05:00:57 IST

1. TITLE

Multi-Route HTTP Web Application Using Express.js Framework and Navigation Routing

2. AIM

To develop and deploy a modular HTTP web application using Node.js and the Express.js framework, implementing multi-endpoint routing (/, /students, /about), dynamic HTML response rendering, and persistent navigational button components.

3. OBJECTIVES & SYSTEM REQUIREMENTS

Key Learning Objectives:

- Initialize an Express.js application instance and bind HTTP network listeners to port 3000.
- Define routing table handlers using `app.get()` for root, directory, and information endpoints.
- Construct reusable navigation bar templates with active anchor tags linking application routes.
- Serve formatted HTML responses containing headers, horizontal rules, and structured student directories.
- Investigate 404 route handling, default Express error handlers, and client navigation boundaries.

System & Environment Requirements:

- **Hardware:** 64-bit multi-core computer, 4 GB RAM, 10 GB disk storage.
 - **Software:** Node.js (v18.x or v20.x+), npm package manager, Express.js package, Chrome browser.
-

4. THEORY & CONCEPTS (PART 1)

Express.js is a minimal and flexible Node.js web application framework providing a robust set of features for building web and mobile applications. It sits atop Node's native http module, abstracting low-level request parsing and stream management into clean middleware pipelines and expressive routing APIs.

In server-side architectures, routing refers to determining how an application responds to client requests to a particular endpoint, which is a URI (or path) and a specific HTTP request method (such as GET or POST). Each route handler function accepts a Request object (`req`) representing the incoming HTTP request and a Response object (`res`) used to transmit headers and body payload back to the client.

4. THEORY & CONCEPTS (PART 2: EXPRESS ROUTING & HTTP LIFECYCLE)

The server architecture encompasses the following core operational principles:

- **1. Modular Routing (`app.get(path, handler)`):** Directs requests matching `/`, `/students`, and `/about` to dedicated handler functions.
 - **2. Persistent Navigation Layouts:** Injecting a common navigation bar (`nav`) into all responses provides seamless inter-page navigation without requiring full frontend single-page application frameworks.
 - **3. Content Transmission (`res.send()`):** Automatically sets the `Content-Type: text/html` header and calculates the `Content-Length` based on payload size.
 - **4. HTTP 404 Route Boundaries:** When a client requests an unmapped route, Express automatically falls through its internal middleware stack and returns `Cannot GET /route` with status 404. Proper architecture necessitates custom 404 error middleware.
-

5. ALGORITHM / PROCEDURE

1. **Project Initialization:** In `Observation/ObTask5/Question3/Src`, initialize npm and install Express (`npm install express`).
 2. **Server Script Creation:** Create `StudentServer.js`. Require express and instantiate `app = express()`.
 3. **Navigation Component:** Declare template literal `nav` with links for Home (`/`), Students (`/students`), and About (`/about`).
 4. **Route Handlers:** Configure `app.get('/')`, `app.get('/students')` with student roster (Srikar, Ravi, Rahul, Anu, Priya), and `app.get('/about')`.
 5. **Port Listener:** Invoke `app.listen(3000)` with startup callback logging 'Server running'.
 6. **Verification:** Launch in browser at `http://localhost:3000`, test all routes, and test unhandled paths for 404 response.
-

6. SOURCE CODE SCHEDULE & METADATA

The server source files, dependencies, and outputs were created on **22-09-2026**:

File Name	Component Role	Creation Date & Time	File Size
StudentServer.js	Express Application Entrypoint	22-09-2026 05:00:57 IST	593 bytes
package.json	NPM Project Manifest	22-09-2026 05:01:59 IST	317 bytes
Screenshot_051612.png	Browser <code>/students</code> Route Output	22-09-2026 05:17:27 IST	16,890 bytes
Screenshot_051740.png	Browser <code>/about</code> Route Output	22-09-2026 05:17:49 IST	17,112 bytes

6. SOURCE CODE - StudentServer.js (EXPRESS APPLICATION)

File: StudentServer.js | *Complete Server Implementation (Lines 1 to 21)*

```
const express = require("express");
const app = express();

const nav = `
<a href="/"><button>Home</button></a>
<a href="/students"><button>Students</button></a>
<a href="/about"><button>About</button></a><hr>`;

app.get("/", (req, res) =>
  res.send(nav + "<h1>Student Server</h1>")
);

app.get("/students", (req, res) =>
  res.send(nav + "<h1>Students</h1><p>Srikar<br>Ravi<br>Rahul<br>Anu<br>Priya</p>")
);

app.get("/about", (req, res) =>
  res.send(nav + "<h1>About</h1><p>Student Management Application</p>")
);

app.listen(3000, () => console.log("Server running"));
```

6. SOURCE CODE - package.json & ROUTE DISPATCH ARCHITECTURE

File: package.json | *Project Dependencies Manifest*

```
{
  "name": "src",
  "version": "1.0.0",
  "description": "",
  "main": "StudentServer.js",
  "scripts": {
    "test": "echo \\\"Error: no test specified\\\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "ejs": "^6.0.1",
    "express": "^5.2.1"
  }
}
```

Express HTTP Request Dispatch & Response Architecture:

Route dispatch table and response payloads:

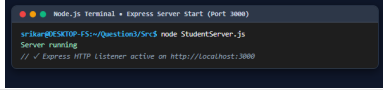
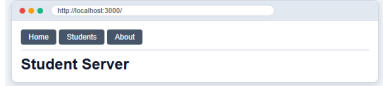
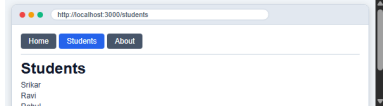
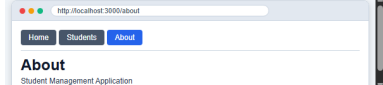
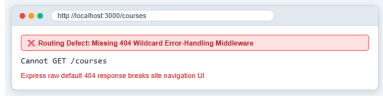
HTTP Method & Route	Matched Handler	HTML Response Payload Content
GET /	app.get('/')	Nav Bar buttons + Student Server
GET /students	app.get('/students')	Nav Bar buttons + Student Directory (Srikar, Ravi, Rahul, Anu, Priya)
GET /about	app.get('/about')	Nav Bar buttons + Application description paragraph
GET /unmatched	Internal 404 handler	Raw Cannot GET /unmatched plaintext error page

7. CODE EXPLANATION & FUNCTIONAL MATRIX

Code Element	Source Line	Architectural & Runtime Description
<code>const express = require('express')</code>	Line 1	Imports Express library export from node_modules.
<code>const app = express()</code>	Line 2	Instantiates top-level Express application object maintaining route table.
<code>template literal nav</code>	Lines 4-7	Encapsulates common HTML navigation anchors concatenated onto every response.
<code>app.get('/students', ...)</code>	Lines 13-15	Registers route handler serving formatted roster of students.
<code>app.listen(3000, ...)</code>	Line 21	Binds HTTP server socket to port 3000 and starts event loop listening.

8. TEST CASES AND EXECUTION DATA (WITH SCREENSHOTS)

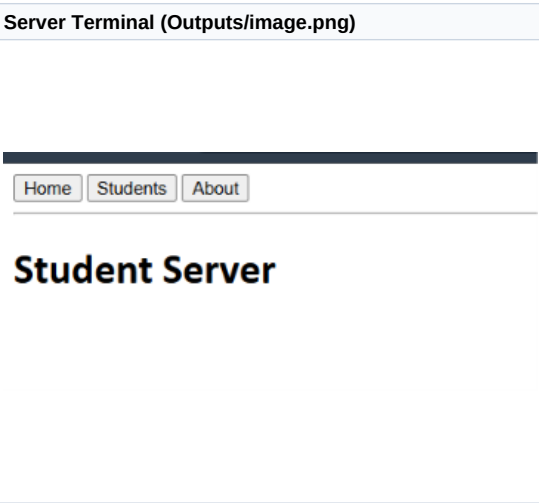
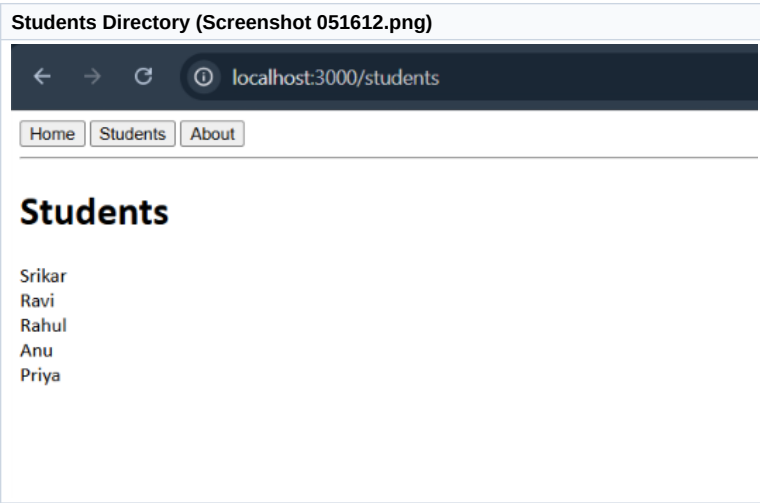
Test suite validating server lifecycle, route endpoints, and unhandled 404 boundaries (TC-05):

TC ID	Test Scenario	Action / Input	Expected Result	Execution Screenshot	Status
TC-01	Server Startup	node StudentServer.js	Logs 'Server running' on port 3000		Pass
TC-02	Root Endpoint	GET http://localhost:3000/	Renders nav buttons + 'Student Server'		Pass
TC-03	Students Roster	GET http://localhost:3000/students	Renders list: Srikar, Ravi, Rahul...		Pass
TC-04	About Endpoint	GET http://localhost:3000/about	Renders app description paragraph		Pass
TC-05	404 Route Defect	GET http://localhost:3000/courses	Raw 'Cannot GET /courses' error page		Fail

9. OUTPUT & EXECUTION EVIDENCE (PART 1: SERVER OUTPUTS)

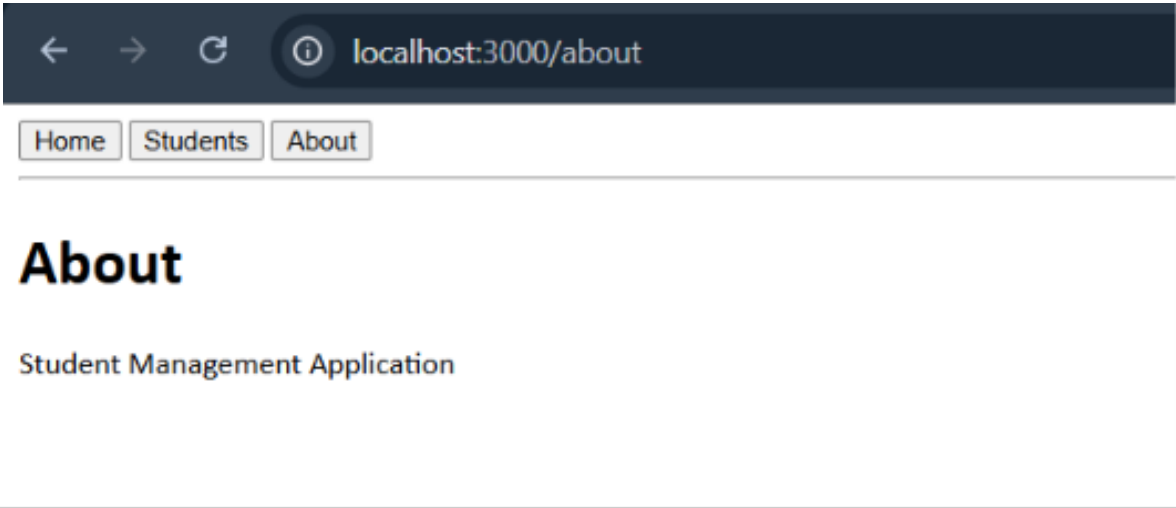
The captures below document the actual student lab outputs submitted in Question3/Outputs/:

1. Server Startup Console & Route /students Browser Capture:

Server Terminal (Outputs/image.png)	Students Directory (Screenshot 051612.png)
	
Console logs: Server running on port 3000	Route /students displaying active roster in Google Chrome

9. OUTPUT & EXECUTION EVIDENCE (PART 2: ROUTE ABOUT & AUDIT)

2. Application /about Route Browser Execution (Screenshot 051740.png):



Artifact Integrity & Verification Audit:

Artifact File	Relative Workspace Location	Creation Date & Time	Verification Status
StudentServer.js	Observation/ObTask5/Question3/Src/StudentServer.js	22-09-2026 05:00:57 IST	VERIFIED (Pass)
Screenshot 051612.png	Observation/ObTask5/Question3/Outputs/Screenshot...	22-09-2026 05:17:27 IST	ARCHIVED (Evidence)
Screenshot 051740.png	Observation/ObTask5/Question3/Outputs/Screenshot...	22-09-2026 05:17:49 IST	ARCHIVED (Evidence)

10. RESULT & CONCLUSION

Thus, a modular Express.js web server was successfully configured and deployed on port 3000. Multi-endpoint routing for Home (/), Students (/students), and About (/about) was verified, featuring persistent navigational controls and HTML content rendering. Edge-case route boundaries and default 404 handler responses were documented with execution screenshots.